

DDO Studio

Program Overview, Capabilities, Boundaries, and Usage Notes

Version	1.7.2
Platform	Windows x64
Primary function	Local DDO model viewing and GLB export
Data source	User-installed local DDO DAT files
Network model	Loopback-only local backend for asset processing

This document describes the current program as implemented. It intentionally omits discarded one-off experiments and does not contain personal identifiers, machine-specific user paths, or local test data.

DDO Studio is an independent third-party project and is not affiliated with or endorsed by Standing Stone Games.

What DDO Studio is

DDO Studio is a local Windows application for inspecting and exporting renderable 3D assets from a user's own Dungeons & Dragons Online installation. It is primarily a model-research, preservation, and hobbyist content tool. The program reads the game's installed DAT files, follows the relationships that connect named game records to model data, reconstructs model geometry and materials, and displays the result in a local 3D viewer.

The current model library is intentionally narrower than the full game database. Normal search includes directly renderable models plus standalone weapons and shields. Worn/composed equipment classes that are not dependable standalone models are kept out of the public model browser. There is no category dropdown for Equipment, Weapons, Armor, or similar lists; the library is one searchable model list.

DDO Studio does not need the game to be running. It does not log into a DDO account, connect to a game world, control a character, or alter gameplay. The application works from files already installed on the local computer and writes its own caches, previews, diagnostics, and exports to user-local folders.

How the model pipeline works

A model begins with a named local game record. DDO Studio follows typed relationships through the client data rather than assuming that every number found in a binary record is a valid model reference. The normal chain is:

```
DbProperties -> PhysObj -> VisualDescription -> Setup -> RenderMesh -> materials/textures
```

The Setup is the key renderable object. Once a valid Setup is resolved, DDO Studio reconstructs mesh primitives, materials, UVs, textures, skeleton information, skinning data, and validated animation data when available. DDO uses a Z-up coordinate system while glTF uses Y-up, so the exporter applies the required coordinate conversion while keeping the skeleton and skinned meshes in the same transform hierarchy.

The desktop viewer uses Three.js inside WebView2. Model transforms and pedestal transforms are separate. Position X/Y/Z and Rotation X/Y/Z affect the model only; pedestal height and scale affect only the pedestal. Weapon records use a presentation default of Position Y 1.40x and Rotation X 90°, with the other transform axes at zero; Reset Model Transform returns Weapons to those defaults.

What can be exported

DDO Studio exports glTF 2.0 GLB files. GLB is useful because it can package geometry, materials, textures, skeletons, and compatible animation data into a single portable model file.

A GLB exported by DDO Studio can be opened in Blender. From Blender, a user can inspect the mesh, pose or clean it, separate parts, repair geometry, add supports or bases, and export the result to an STL file. That makes DDO Studio useful for hobbyist workflows that turn locally extracted DDO models into printable geometry for personal 3D-printing projects.

STL does not preserve materials, textures, rigs, or animation; it is primarily a triangle-mesh format. For that reason, GLB is the better archival/editing format and STL is usually the final manufacturing format after the model has been prepared in Blender or another mesh editor.

Technical export capability is not a grant of intellectual-property rights. DDO assets remain subject to the rights of their respective owners, so redistribution or commercial use should be considered separately from the technical ability to export a model.

Animations

DDO Studio can discover and decode supported older Havok animation records from the local Anim DAT. Decoding a record is not enough by itself to make it playable: the program also checks whether the clip is compatible with the loaded skeleton and available binding evidence.

Animation defaults are compatibility-gated. For Weapon records (weenieType 0x00020081), DDO Studio prefers 0x05005943 and enables looping when that clip is present in the validated compatible list. If that Weapon clip is unavailable, 0x05000440 is the fallback when compatible. Other models prefer compatible 0x05000440. Users can also rename anonymous animation IDs. Those local overrides are stored at:

```
%LOCALAPPDATA%\DDO Asset Studio\animation-aliases.json
```

The alias file is plain JSON. Version 1.7.2 also ships a curated built-in name map in viewer/animation-names.json; LocalAppData aliases override the built-in label for matching IDs and are not bundled into release packages.

Local-only design and network boundary

The current source is designed around local files. The desktop application launches a small ASP.NET Core backend bound to the loopback interface (127.0.0.1) and the exporter talks to that local backend. The Three.js viewer is also served through local application plumbing. The model pipeline does not require a remote DDO service.

The program has no DDO login flow, no account credential handling, no packet sniffer, no process injection system, no remote-control mechanism, and no code for sending game commands to a DDO server. It does not need to attach to a running DDO process.

Because the program works on the local client data, it can only know what is present in those local files. It cannot query live server state, account state, current instances, live drops, another player's private data, or server-only logic. Some client DAT records contain static treasure references; if those references are encountered by the local parser, they are still client-side data and are not access to live or server-side loot tables.

Can DDO Studio hack the game?

No. DDO Studio is not a game hack in the technical sense. It does not inject code into the DDO client, patch the running process, modify memory, intercept packets, impersonate the client, automate combat, move a character, generate items, alter character stats, or send commands to the game server.

It is an offline/local asset reader and exporter. Its meaningful output is things such as preview models, GLB files, images, diagnostics, and locally stored animation labels.

Can it see server-side information or loot?

No server-side information is available to DDO Studio through its model pipeline. It has no server connection or authentication mechanism. It cannot inspect a server's live loot rolls, drop chances, instance state, account inventory, economy data, or unreleased server-only content.

The distinction matters: a local game installation can contain static data structures that happen to mention treasure or item relationships. Reading those local structures is not the same as seeing the server's current loot tables or server-side drop logic, and DDO Studio does not have a mechanism to ask the server for that information.

Does DDO Studio violate DDO's Terms of Service or Code of Conduct?

Under a strict reading of Standing Stone Games' published DDO Code of Conduct, DDO Studio should be treated as an unauthorized third-party utility unless Standing Stone Games has given express written permission. The Code of Conduct states that players may not modify the game client and may not create, post, or distribute utilities, emulators, or other third-party software tools without express written permission from SSG.

So the practical answer is yes: assume use or distribution carries Terms/Code-of-Conduct risk unless SSG explicitly authorizes it. That policy conclusion is separate from the technical behavior of the program. DDO Studio does not have to modify the DDO client or interact with live gameplay to fall within broad language covering third-party utilities.

This document is not legal advice, and only Standing Stone Games can give an authoritative determination about whether a specific build or use is permitted.

Comparison with Dungeon Helper

Dungeon Helper is a useful comparison because it is another well-known third-party DDO tool, but its architecture is different. Dungeon Helper describes itself as a plugin hosting framework that enables plugins to do things "inside DDO." Its current public documentation says version 4 runs with Administrator permissions, supports the 64-bit client, uses an in-game overlay, and is intended to run alongside DDO.

DDO Studio is less coupled to the live client. It does not need DDO running, does not need an in-game overlay, does not require an administrator-level companion process for normal asset viewing, and does not host gameplay plugins. It reads installed DAT files, starts a loopback-only local backend, renders models in its own viewer, and exports files to disk.

That makes DDO Studio technically less intrusive into a live game session than a plugin framework that operates alongside the running client. It does **not** automatically make DDO Studio approved under SSG policy. Dungeon Helper's own FAQ says Dungeon Helper itself does not break the DDO EULA/ToS and that potential issues come from individual plugins. That is Dungeon Helper's stated position; SSG remains the authority on its own rules.

Functional comparison

Area	DDO Studio	Dungeon Helper
Primary purpose	Local asset viewing/export	Plugin hosting framework for DDO
DDO must be running	No	Designed to run alongside DDO
In-game overlay	No	Yes
Administrator requirement	Not required for normal asset workflow	Public docs say version 4 requires Administrator
Reads local DAT assets	Yes	Uses its own plugin/SDK model
Exports GLB models	Yes	Not its primary purpose
Blender / STL workflow	Yes, through exported GLB	Not its primary purpose
Sends gameplay commands	No	Depends on plugin behavior; framework hosts plugins
Server/account login	No	Not the purpose described by its public FAQ
Policy status	Treat as unauthorized unless SSG grants permission	Maintainers state framework itself does not violate; plugins can create issues

Privacy and what gets written to disk

DDO Studio does not need a DDO account name, password, character login, or server token. Runtime files are stored under the current Windows user's LocalAppData area, including the model index/cache, preview GLBs, diagnostics, WebView2 data, and animation aliases. Manual exports go wherever the user selects.

The source/release packaging process excludes LocalAppData state, user aliases, preview caches, logs, screenshots, PDB debug symbols, and machine-specific build paths. The distributable source contains program code and documentation, not a copy of the user's DDO DAT files.

How the current program architecture was reached

The project developed in stages. First, the local DAT files were made reliably readable through typed object ranges and a small local API. Next, the model relationship chain was narrowed to direct evidence from named records through PhysObj, VisualDescription, Setup, and RenderMesh. Once model selection became dependable, the exporter was built around glTF 2.0 so preview and permanent export used the same geometry/material pipeline.

Material and texture reconstruction followed, then skeleton and skin-weight export. Animation work was deliberately split into discovery, decoding, and compatibility so an anonymous animation record would not be treated as valid merely because it happened to have a plausible track count. The viewer then gained model inspection, independent XYZ position/rotation, environment import, animation browsing, and scene presentation controls.

The final model-library boundary is intentionally conservative: directly renderable objects are searchable, standalone weapons and shields remain available, and worn/composed equipment that does not behave like a complete standalone model is not exposed as a normal model result. The result is a focused local model tool rather than a general-purpose browser for every record in the game's databases.

Basic FAQ

Does DDO Studio change my DDO installation? No. The intended workflow is read-only against the selected DDO DAT files. DDO Studio writes its own cache/preview/diagnostic files under LocalAppData and writes exports to locations selected by the user.

Do I need to launch DDO first? No. DDO Studio works directly from the installed local data files.

Can I export a model for Blender? Yes. Export the model as GLB and import that GLB into Blender.

Can I 3D print a DDO model? Technically, yes. A common workflow is DDO Studio -> GLB -> Blender cleanup/pose -> STL -> slicer. Intellectual-property rights remain separate from the technical workflow.

Can it give me items, platinum, XP, or stats in-game? No. It has no mechanism to change server-authoritative game state.

Can it reveal live drop rates or server loot rolls? No. It has no server connection. Any treasure-related data that may exist in local DATs is static client data, not live server-side information.

Can it see other players or my account? No. It does not authenticate to DDO or query the live game service.

Where are my custom animation names?

```
%LOCALAPPDATA%\DDO Asset Studio\animation-aliases.json
```

References

- Standing Stone Games, *Dungeons & Dragons Online - Code of Conduct*, updated December 20, 2023: <https://help.standingstonegames.com/hc/en-us/articles/24334942930579-Dungeons-Dragons-Online-Code-of-Conduct>
- Dungeon Helper, *Frequently Asked Questions*: <https://dungeonhelper.com/frequently-asked-questions/>
- Dungeon Helper, *Home / Installation Guide*: <https://dungeonhelper.com/>

Reference pages were reviewed September 2026. Policies and third-party-tool documentation can change; check the current pages before relying on them.